

台灣國道旅行時間可靠度分析 & 旅行中位數即時運算分析

BDDE38 第三組 | A、B、C、D、E

團隊介紹

K8s 湖倉建置 & 爬蟲



A

資料蒐集・環境建置

網路爬蟲
K8s / 湖倉 /
Spark / Iceberg



B

基礎架構・資源調校

K8s / 湖倉
Spark / Iceberg
運算參數調校

資料工程 & 分析



C

高速公路可靠度分析

資料預處理
資料運算與分析
資料輸出與前端串接



D

旅行時間中位數預測

資料預處理
資料運算與分析
資料輸出與前端串接

前端 API 串接



E

視覺化

前端介面開發
地圖圖表
資料視覺化

研究動機(專題簡介)

星期五晚上 6 點,交流道前,地圖是綠的。

上了國道,時速卻只剩 20——要不要上去,你怎麼決定?

猜

目前駕駛唯一的決策工具



通勤者在猜

晚 10 分鐘出門,會不會其實一樣時間到公司?



物流業者在猜

這條路這個時段穩不穩?猜錯,整車的貨都延誤。



旅客在猜

知道會塞,但塞 20 分鐘跟 1 個半小時是兩回事。

研究目的(商業價值)

先想清楚:痛點不是「塞車」,是「**時間不確定性**」。

塞車但穩定,大家願意走;不塞但不穩定,才讓人焦慮。所以我們要從歷史資料回答兩個問題:

核心問題

「這段路到底
要不要多抓時間？」

從用路人、物流、旅遊安排
的共同痛點出發



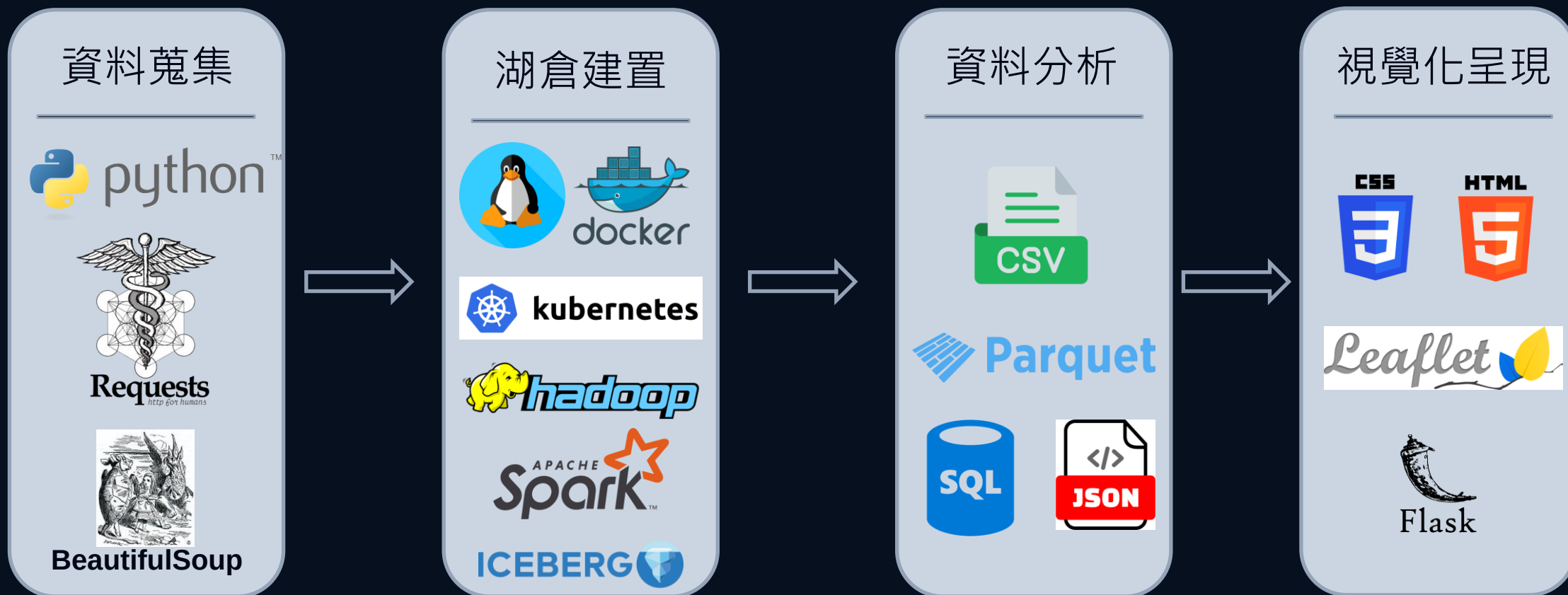
① 分析一：可靠度分析

依據日別、時段做篩選，
找出旅行時間最不穩定的
路段，分析「哪些高速公路
路段的旅行時間最難預估」。

② 分析二：即時中位數查詢

依起訖點、星期、時段與
假日條件，提供歷史旅行
時間查詢與行程預估。

研究流程 & 使用工具



Web Scraping & ELT

爬蟲和ELT

團隊介紹

動機與目的

研究工具

ETL 資料工程

環境建置

分析與效能

視覺化成果

資料來源

本專題利用交通部高公局交通資料庫之 M04A 作為歷史資料來源，分析旅行時間可靠度，並提供自訂條件計算兩地間行車時間的功能，協助使用者降低交通不確定性，提升出行規劃效率。

主要資料: 站間各車種中位數旅行時間 (M04A)

時間區間: 2021/06/22 ~ 2026/06/30

總資料量: 1,351,526,400(筆)



交通部高速公路局
FREEWAY BUREAU, MOTC

交通資料庫

站間各車種中位數旅行時間 (M04A) - 欄位說明

TimeStamp	GantryFrom	GantryTo	VehicleType	TravelTime	Volume
2018/08/20	01F0017N	01F0005N	31	46	44
2018/08/20	01F0017N	01F0005N	32	45	16
2018/08/20	01F0017N	01F0005N	41	49	2
2018/08/20	01F0017N	01F0005N	42	0	0
2018/08/20	01F0017N	01F0005N	5	0	0

TimeStamp :

報表時間戳記，以下游門架時間戳記代表，此外即時與更新資料時間戳記代表的時間區間不同

- 即時資料：2024-01-01 10:00 代表時間區間為 2024-01-01 09:55~10:00

- 更新資料：2024-01-01 10:00 代表時間區間為 2024-01-01 10:00~10:05

GantryFrom :

上游門架編號，門架所在路段詳使用手冊

GantryTo :

中位數旅行時間，計算單位為秒

VehicleType :

車種，31小客車、32小貨車、41大客車、42大貨車、5聯結車

TravelTime :

依車種統計相鄰上下游偵測站之中位數旅行時間

Volume :

交通量，時間區間內經過上下游門架之車流量總量

特徵整理

TimeStamp	GantryFrom	GantryTo	VehicleType	TravelTime	Volume
時間戳記	上游門架編號	下游門架編號	車種： 31小客車、32小貨車、41大客車、42大貨車、5聯結車	中位數旅行時間	交通量

	A	B	C	D	E	F
1	2026/6/23 18:45	01F0017N	01F0005N	31	49	81
2	2026/6/23 18:45	01F0017N	01F0005N	32	48	16
3	2026/6/23 18:45	01F0017N	01F0005N	41	50	14
4	2026/6/23 18:45	01F0017N	01F0005N	42	0	0
5	2026/6/23 18:45	01F0017N	01F0005N	5	0	0
6	2026/6/23 18:45	01F0029N	01F0017N	31	49	116
7	2026/6/23 18:45	01F0029N	01F0017N	32	47	22
8	2026/6/23 18:45	01F0029N	01F0017N	41	50	21
9	2026/6/23 18:45	01F0029N	01F0017N	42	0	0
10	2026/6/23 18:45	01F0029N	01F0017N	5	0	0

國道1號			
南向		北向	
門架	路段	門架	路段
01F0005S	基隆端-基隆	01F0005N	基隆端-基隆
01F0017S	基隆-八堵	01F0017N	基隆-八堵
01F0029S	八堵-大華系統	01F0029N	八堵-大華系統
01F0061S	大華系統-五堵	01F0061N	大華系統-五堵

特徵整理

高速公路局交通資料庫

歷史資料庫檔案下載 上一頁 歷史資料庫		
20260529/	2026-06-01 04:13	-
20260528/	2026-05-31 03:42	-
20260527/	2026-05-30 03:42	-
20260526/	2026-05-29 03:42	-
20260525/	2026-05-28 03:43	-
20260524/	2026-05-27 04:13	-
20260523/	2026-05-26 04:14	-
tt2	2022-11-16 09:12	48
tt1	2022-11-16 09:12	48
M04A_20260522.tar.gz	2026-06-26 05:00	4.0M
M04A_20260521.tar.gz	2026-06-25 05:00	3.9M
M04A_20260520.tar.gz	2026-06-24 05:00	3.9M
M04A_20260519.tar.gz	2026-06-23 05:00	3.9M
M04A_20260518.tar.gz	2026-06-22 05:00	3.9M

```
bigred@bigred-Group3:~/raw/M04A/20260630$ tree
.
├── 00
│   ├── TDCS_M04A_20260630_000000.csv
│   ├── TDCS_M04A_20260630_000500.csv
│   ├── TDCS_M04A_20260630_001000.csv
│   ├── TDCS_M04A_20260630_001500.csv
│   ├── TDCS_M04A_20260630_002000.csv
│   ├── TDCS_M04A_20260630_002500.csv
│   ├── TDCS_M04A_20260630_003000.csv
│   ├── TDCS_M04A_20260630_003500.csv
│   ├── TDCS_M04A_20260630_004000.csv
│   ├── TDCS_M04A_20260630_004500.csv
│   ├── TDCS_M04A_20260630_005000.csv
│   └── TDCS_M04A_20260630_005500.csv
```

特徵整理

```
bigred@ubuntu-server: ~/dataset/csv_1/year=2026/month=06$ cat data.csv | head -5
2026/06/01 00:00,01F0017N,01F0005N,31,44,19
2026/06/01 00:00,01F0017N,01F0005N,32,42,3
2026/06/01 00:00,01F0017N,01F0005N,41,0,0
2026/06/01 00:00,01F0017N,01F0005N,42,50,1
2026/06/01 00:00,01F0017N,01F0005N,5,0,0
bigred@ubuntu-server: ~/dataset/csv_1/year=2026/month=06$ cat data.csv | tail -5
2026/06/30 23:55,05F0438N,05FR143N,31,104,2
2026/06/30 23:55,05F0438N,05FR143N,32,0,0
2026/06/30 23:55,05F0438N,05FR143N,41,0,0
2026/06/30 23:55,05F0438N,05FR143N,42,0,0
2026/06/30 23:55,05F0438N,05FR143N,5,0,0
```

特徵整理

```
bigred@bigred-Group3:~/dataset/parquet_1$ tree
```

```
.
├── year=2021
│   ├── month=06
│   │   └── data.parquet
│   ├── month=07
│   │   └── data.parquet
│   ├── month=08
│   │   └── data.parquet
│   ├── month=09
│   │   └── data.parquet
│   ├── month=10
│   │   └── data.parquet
│   ├── month=11
│   │   └── data.parquet
│   └── month=12
│       └── data.parquet
└── year=2022
    ├── month=01
    │   └── data.parquet
    └── month=02
        └── data.parquet
```

```
bigred@bigred-Group3:~/dataset/parquet_1/year=2026/month=06$ duckdb -c "SELECT * FROM 'data.parquet' LIMIT 5"
```

year int64	month int64	date int32	weekday int8	time varchar	route varchar	vehicle_type int16	travel_time int32	volume int32
2026	6	1	1	00:00	01F0017N_01F0005N	31	44	19
2026	6	1	1	00:00	01F0017N_01F0005N	32	42	3
2026	6	1	1	00:00	01F0017N_01F0005N	41	0	0
2026	6	1	1	00:00	01F0017N_01F0005N	42	50	1
2026	6	1	1	00:00	01F0017N_01F0005N	5	0	0

Infrastructure & Big Data Platform

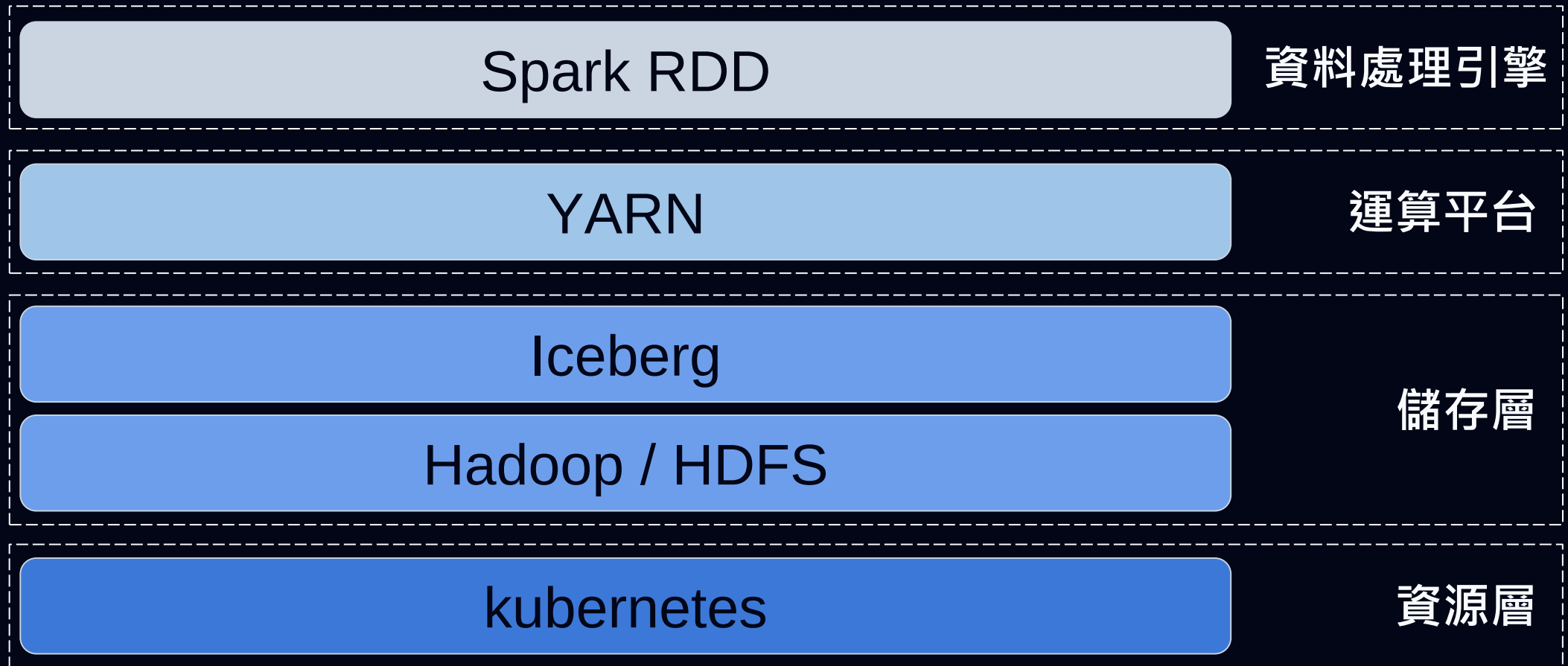
大數據湖倉平台

傳統資料庫與資料湖倉比較

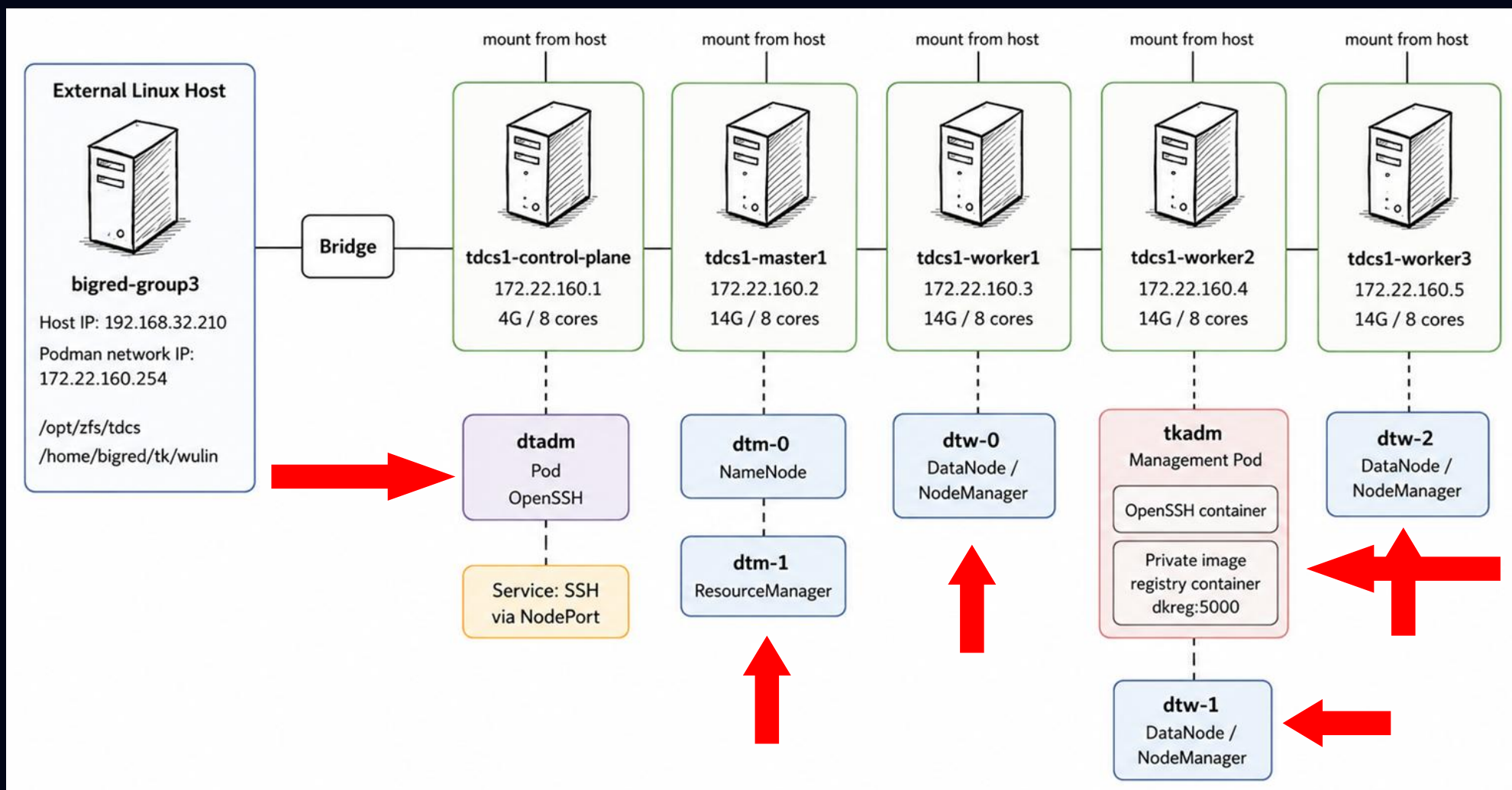
	 Database	 Lakehouse
修改方式	Row Update (原地修改)	Data File, Delete File, Snapshot (建立新版本)
修改速度	Transaction 快	CoW 較慢 / MoR 較快
查詢速度	小量精準查詢快 (毫秒級)	大量掃描分析快
擴展方式	cale Up (垂直)	Scale Out (水平)
適合工作	OLTP (交易型)	OLAP (分析型)
優勢應用場景	訂單、會員、庫存等高頻低延遲交易	歷史資料、日誌、大數據彙總與分析

 **核心結論：**傳統資料庫擅長即時交易 (OLTP)，Lakehouse 擅長大數據分析 (OLAP)。兩者不是互相取代，而是解決不同的問題。

主要技術元件



大數據湖倉平台建置



Reliability Analysis

可靠度分析

可靠度關鍵定義：Buffer Index越高，代表路況越不穩定

可靠度低不等於常塞車；可靠度看「路段行駛時間靠不靠譜、能不能預測」。

$$\text{Buffer Index (BI)} = \text{Buffer Time} / \text{Median}$$

P50

一般情況下的典型旅行時間：
平常大概需要多久時間。

P95

最壞情境旅行時間：保守需要
緩衝的時間。

Buffer Time (P95 - P50)

額外預留時間：看最壞旅行時間
比典型旅行時間差多久。

穩定 $BI < 0.25$

平常旅行時間與最壞的旅行時間差不多

普通 $0.25 \leq BI < 0.50$

最壞旅行時間比平常旅行時間再多一點

不穩定 $0.50 \leq BI < 1.00$

要比平常旅行時間預估多一倍的時間

高風險 $BI \geq 1.00$

要比平常旅行時間預估多一倍以上的時間

Reference: Pu, Wenjing. (2011). Analytic Relationships Between Travel Time Reliability Measures. Transportation Research Record: Journal of the Transportation Research Board. 2254. 122-130. 10.3141/2254-13.

可靠度分析_四種分析維度

時段維度分析

時段 | 時間

凌晨 | 00:00–05:59

早尖峰 | 06:00–08:59

日間 | 09:00–15:59

晚尖峰 | 16:00–18:59

夜間 | 19:00–23:59

日別維度分析

平日 | Weekday

假日 | Weekend

整體維度分析

不分日期

不分時段

時段 X 日別交叉分析

平日 x 凌晨

平日 x 早尖

峰

平日 x 日間

平日 x 晚尖

峰

平日 x 夜間

假日 x 凌晨

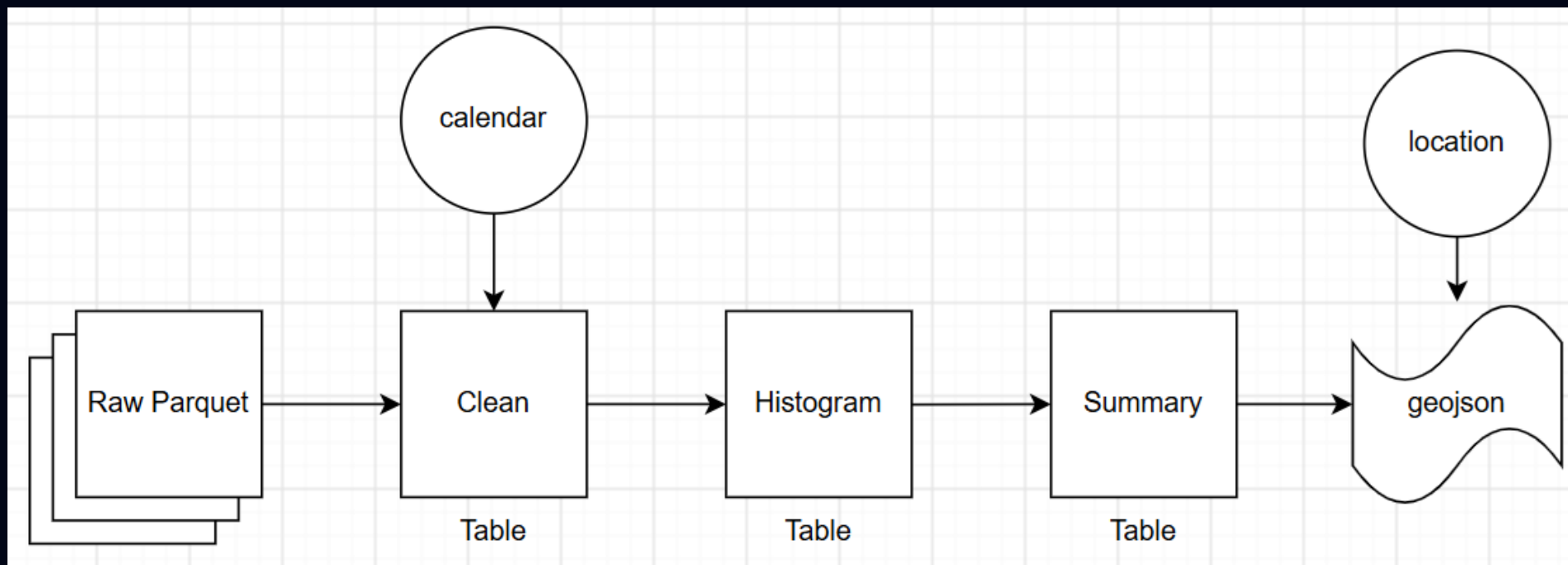
假日 x 早尖峰

假日 x 日間

假日 x 晚尖峰

假日 x 夜間

可靠度分析_流程架構圖



```
spark-sql ()> SHOW NAMESPACES;
tdcs
Time taken: 0.056 seconds, Fetched 1 row(s)
spark-sql ()> SHOW TABLES IN tdcs;
m04a_histogram
m04a_reliability_summary
m04a_clean
Time taken: 0.024 seconds, Fetched 3 row(s)
```

可靠度分析_warehouse資料結構

```
bigred@dtadm:~$ hdfs dfs -ls /user/bigred/warehouse/tdcs_iceberg/tdcs
Found 3 items
drwxr-xr-x  - bigred bigboss      0 2026-07-08 17:51 /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_clean
drwxr-xr-x  - bigred bigboss      0 2026-07-08 21:04 /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_histogram
drwxr-xr-x  - bigred bigboss      0 2026-07-09 18:48 /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_reliability_summary
bigred@dtadm:~$ hdfs dfs -ls /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_clean
Found 2 items
drwxr-xr-x  - bigred bigboss      0 2026-07-08 17:50 /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_clean/data
drwxr-xr-x  - bigred bigboss      0 2026-07-08 17:51 /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_clean/metadata
```

可靠度分析_warehouse資料結構



Iceberg 查詢時無須直接掃描 HDFS 中的所有檔案，而是先讀取 Metadata，快速定位需要的 Parquet 檔案，提升查詢效率。同時，每次資料寫入都會產生新的 Snapshot，保留資料表的歷史狀態，支援版本回溯。

```
bigred@dtadm:~$ hdfs dfs -ls /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_clean/metadata
```

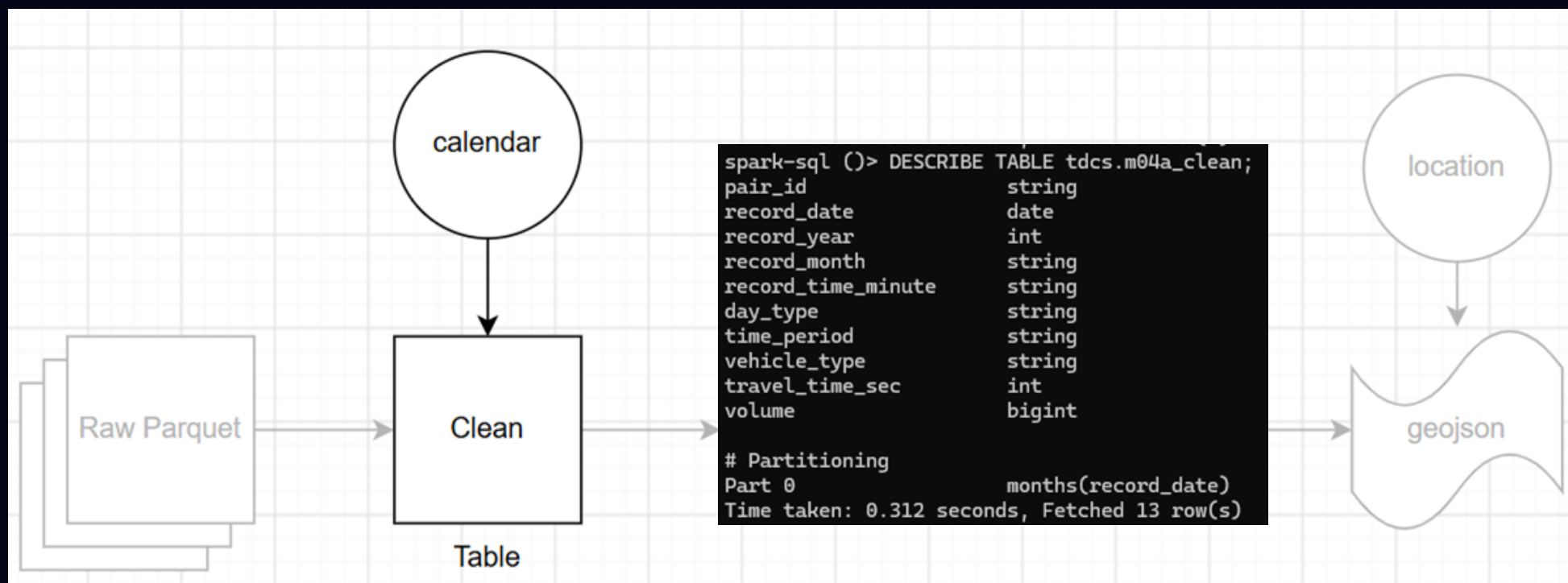
```
Found 4 items
```

```
-rw-r--r-- 3 bigred bigboss 14001 2026-07-08 17:51 /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_clean/metadata/545a1cb6-80e5-47e9-97fc-6d19e45b9ad4-m0.avro
-rw-r--r-- 3 bigred bigboss 4476 2026-07-08 17:51 /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_clean/metadata/snap-275204966352698987-1-545a1cb6-80e5-47e9-97fc-6d19e45b9ad4.avro
-rw-r--r-- 3 bigred bigboss 2317 2026-07-08 17:51 /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_clean/metadata/v1.metadata.json
-rw-r--r-- 3 bigred bigboss 1 2026-07-08 17:51 /user/bigred/warehouse/tdcs_iceberg/tdcs/m04a_clean/metadata/version-hint.text
```

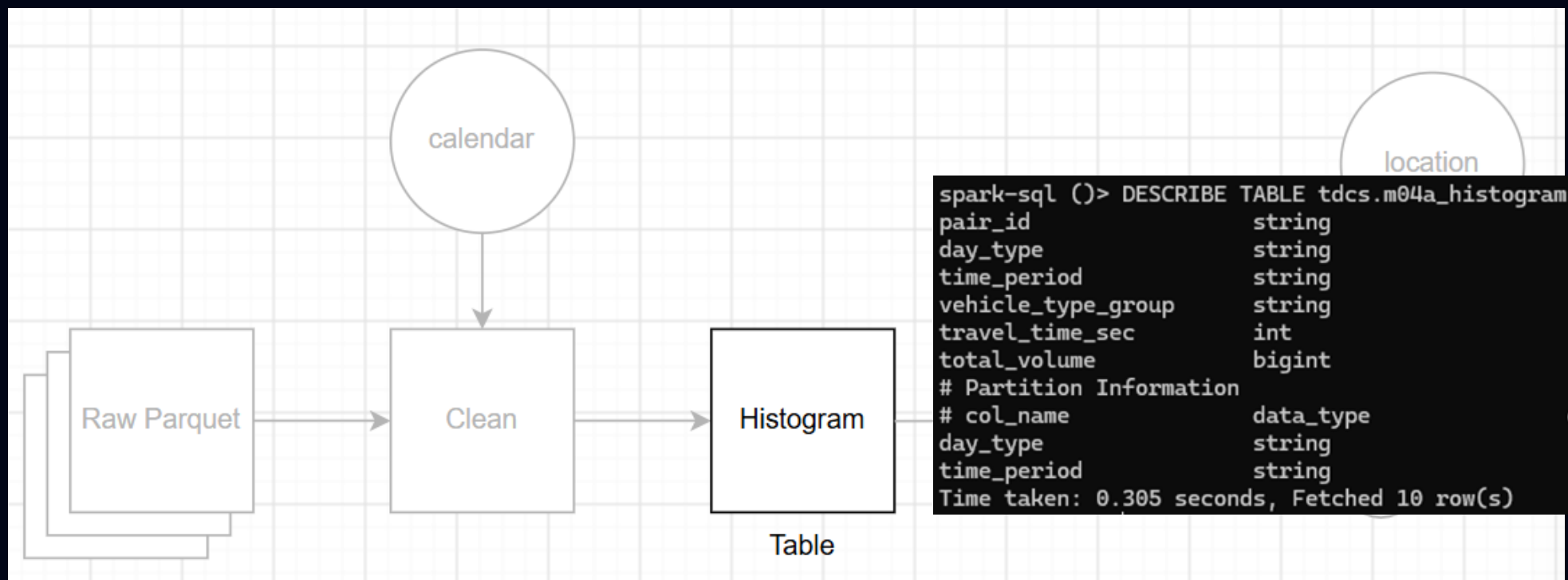
Manifest File

Manifest List

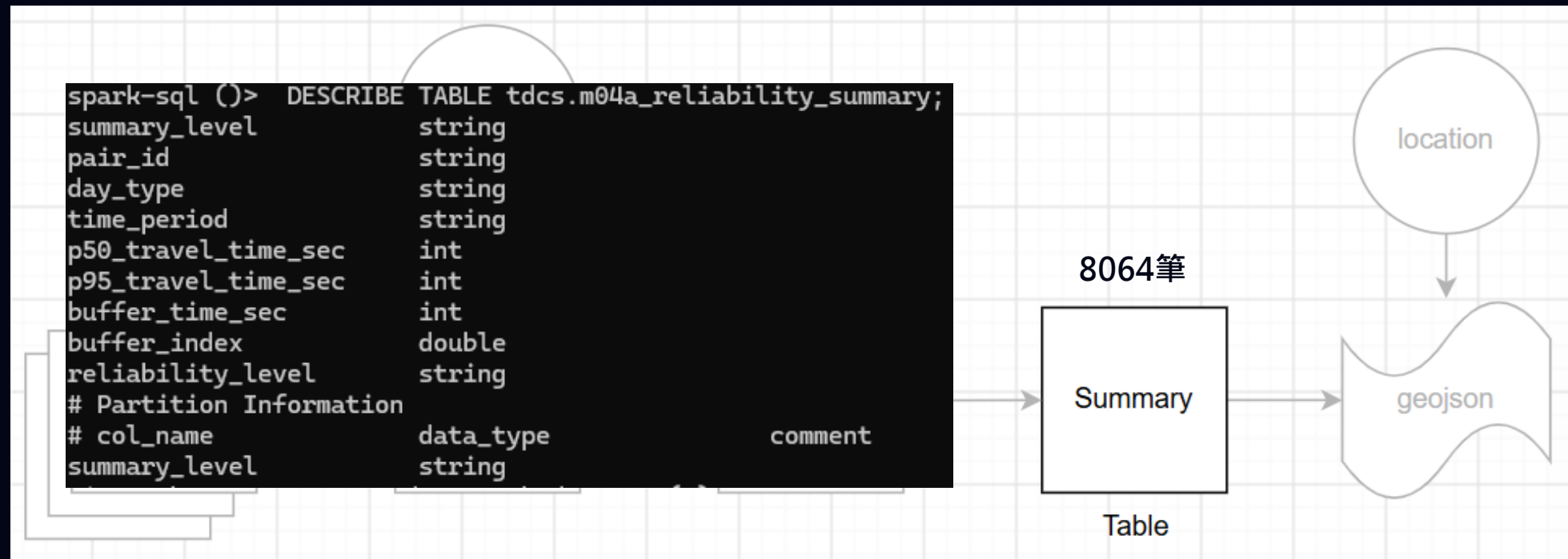
可靠度分析_clean



可靠度分析_histogram



可靠度分析_summary



可靠度分析_GeoJSON



1

資料關聯與整合

將「可靠度分析結果」與「路段經緯度資料」進行 Join，產生包含完整空間資訊的 GeoJSON 資料，作為視覺化的基礎底底。

ACTION

```
summary.join(...)
```



2

Spark 預設輸出

因 Spark 採用分散式架構，Task 會各自平行寫入。因此預設輸出路徑會建立為「資料夾」，而非單一實體檔案。

FOLDER CONTENT

```
_SUCCESS (0 Bytes)  
part-xxxxx (資料檔)
```



3

強制單一分區

為避免產生零碎的 Part 檔，在執行匯出前，強制將 DataFrame 合併為單一 Partition，確保 Spark 最終只會產出一個實體資料檔。

SPARK TRANSFORMATION

```
.coalesce(1)
```



4

檔案更名與提取

寫入完成後，直接從輸出資料夾中提取該唯一的 Part 檔案，移動路徑並重新命名，即可提供給前端進行地圖視覺化。

FINAL OUTPUT FILE

```
reliability_map.geojson
```

解決 Iceberg Writer 錯誤：資料排序策略

⚠ 錯誤根源：Clustered Writer 的連續性限制

Iceberg 預設採用 **Clustered Writer**，要求同一個 Spark Task 中，屬於相同 Iceberg Partition 的資料必須連續出現。當 Writer 切換至下一個分區後，前一個分區的檔案會被關閉；若後續再次出現屬於已關閉分區的資料，就會觸發 **Incoming records violate the writer assumption** 錯誤。

⚡ 策略一：啟用 Fanout Writer

應用場景：Clean Table

- ▶ **分區依據**：依 months(record_date) 進行分區。
- ▶ **運作機制**：針對此表啟用 Fanout Writer，使 Spark 可以同時維持並開啟多個月份分區的 Writer。
- ▶ **實作效益**：即使不同月份的資料交錯到達，也能直接寫入正確分區，免除預先排序的運算成本。

↓ 策略二：資料預先分佈與排序

應用場景：Histogram Table

- ▶ **分區依據**：依 day_type 與 time_period 分區。因 groupBy 與 Shuffle 後的資料不保證連續排列，故採兩階段處理。
- ▶ **Step 1：重新分佈**
以 repartition(day_type, time_period) 將相同分區組合分配至相同的 Spark Partition。
- ▶ **Step 2：分區內排序**
透過 sortWithinPartitions(...) 確保每個 Spark Partition 內的資料皆按照 Iceberg 分區連續排列，避免已關閉分區再次收到資料的錯誤。

Spark Shuffle 運行機制解析

Shuffle

當執行 `groupBy()`、`join()`、`repartition()`、`orderBy()` 等操作時，Spark 會將分散在不同節點上、由不同 Executor 處理的資料，透過 Shuffle 重新搬移與分組。

Hash

執行 `repartition()` 會根據指定欄位計算 Hash，決定每筆資料應進入哪一個 Spark Partition。

ex. `repartition("A", "B")` 會將 A,B 組合視為一組 Key，而相同 Key 會進入相同的 Spark Partition，但不保證 Spark Partition 內部資料之排序，所以後續仍需要執行 `sortWithinPartitions()`。



Shuffle 是 Spark 運算中**資源消耗**的步驟。它涉及跨節點的網路 I/O 傳輸、大量的磁碟讀寫 (Spill)、記憶體緩衝區壓力、Task 排程，以及頻繁的資料序列化與反序列化。

Shuffle 執行流程

1

Hash

Spark 根據 Key 計算 Hash (一個數值)，判斷每筆資料應送往哪一個 Shuffle Partition

2

暫存

原 Executor 先將資料暫存在記憶體或本機磁碟的 Shuffle File。

3

Shuffle Fetch

新 Task 透過網路讀取各 Executor 上屬於自己的 Shuffle Block。

4

分組與聚合

資料集中後，新 Task 再執行分組、Join、排序或聚合。

Shuffle 效能調優與參數配置

Shuffle 關鍵效能配置 (AQE)

設定參數項目

優化與功能運作機制

`spark.sql.shuffle.partitions`

建議值: 60

調整 Shuffle 初始分區數，用以精準控制後續運算的平行度與產生的 Task 數量。

`spark.sql.adaptive.enabled`

預設值: TRUE

啟用 AQE (Adaptive Query Execution)。Spark 會根據實際 Shuffle 資料量與執行統計，動態調整後續執行計畫，而不是完全依賴執行前的估算。

`spark.sql.adaptive.coalescePartitions.enabled`

預設值: TRUE

允許 AQE 自動合併連續且過小的 Shuffle Partition，減少過多零碎 Task 所產生的排程與管理成本。

Task

每一個新的 Spark Partition 通常會交給一個 Task 處理
(Shuffle Partition 數量 = 該 Shuffle Stage 的 Task 數量)

不同階段的shuffle.partitions設定

60

PART

Histogram 階段

Clean Data 初始資料量最龐大，且 groupBy 的欄位與資料維度多，故配置最高分區數。

32

PART

Summary 階段

已由前一階段的 Histogram 進行初步聚合，核心資料量顯著降低，故調降分區數。

16

PART

GeoJSON 輸出

主要處理 Join 與最終地圖格式輸出，整體資料量最小，因此採用最輕量化配置。

數據處理與系統架構優化決策



Volume 型態與防溢決策

Int (32-bit)
約 21.47 億

Long (64-bit)
 9.22×10^{18}

⚠️ **累積溢位威脅**：單筆車流量(volume)不大，但 Histogram 會將五年內相同路段、日別、時段及旅行時間的車流量加總。累積結果可能超過 Int 的上限，且未來新增資料後數值仍會持續增加。因此使用 Long 可避免整數溢位，確保後續 P50、P95 計算正確。

🔗 **自動提升機制**：若 Clean Data 的 Volume 欄位使用 Int，Spark 3.4.4 在執行聚合加總時，會自動將 Int 的加總結果提升為 LongType，不需另外手動轉型。

`F.sum("volume").alias("total_volume")`

💾 **儲存容量權衡**：Long 每筆占用 8 bytes (Int 的 2 倍)，但搭配 Parquet 欄式儲存，實際檔案大小不一定會增加一倍。



程式架構的設計優化

整合 Level 層級
4分析粒度

最終資料整合
共計 8,064 筆

🧩 **降低重複程式與維護成本**：不必在四個 Level 中重複撰寫 Buffer Index 與分類邏輯。

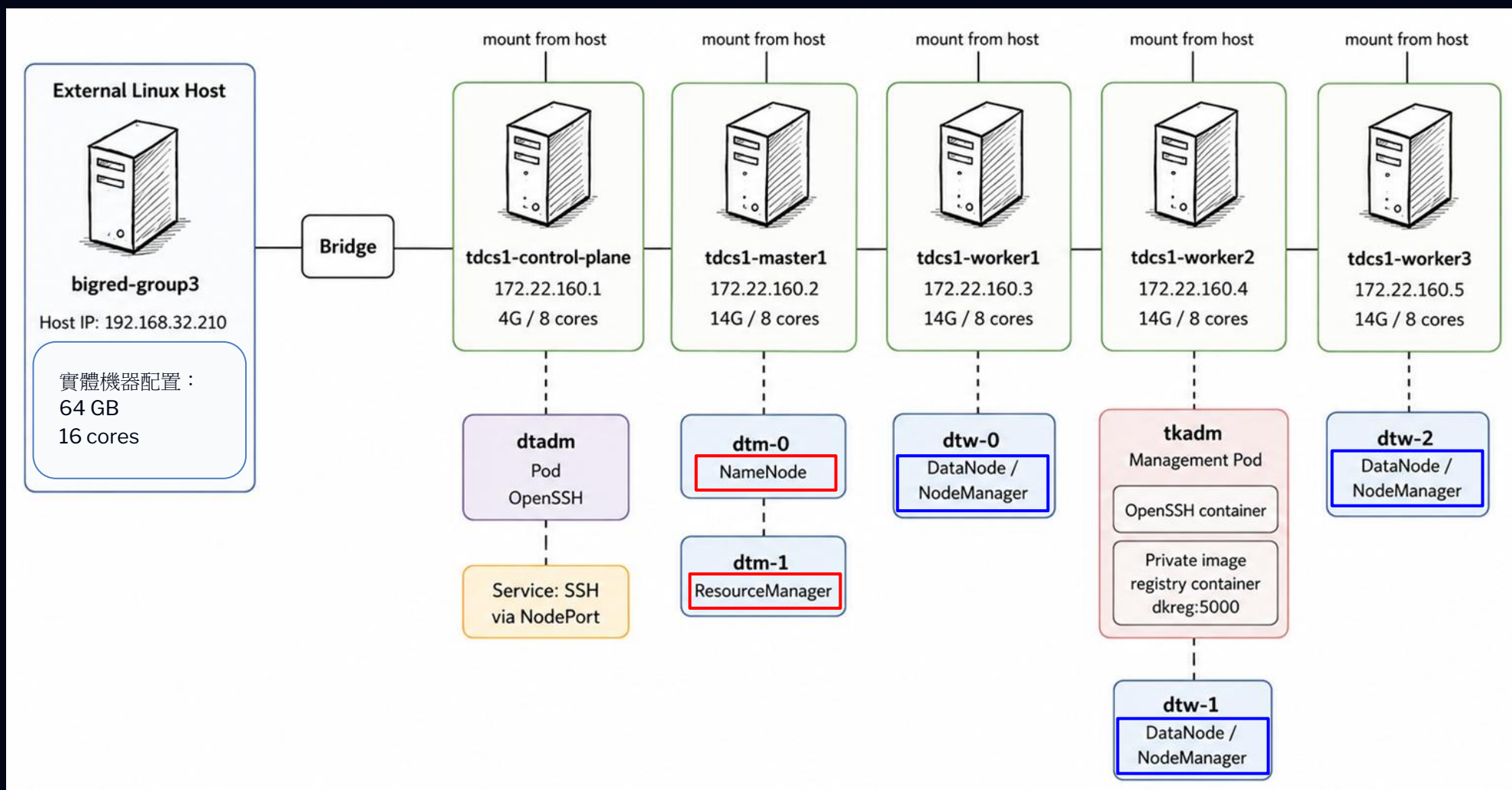
⚖️ **確保計算方法與判斷標準一致**：四種分析層級全部使用相同公式與門檻，避免某個 Level 的計算條件不同。

🔑 **方便後續維護與調整**：未來若修改可靠度門檻，只需修改 UNION 後的一段程式。

SPARK RDD Optimization

Spark 運算參數設定與效能優化

大數據湖倉平台建置



測試機 1

初次執行 `clean.py` 時，叢集的初始設定限制了運算潛力：

yarn-site.xml 參數	設定值	spark-default.conf 參數	設定值
yarn.nodemanager.resource.memory-mb	12288 (12G)	spark.yarn.am.memory	1024m
yarn.nodemanager.resource.cpu-vcores	8	spark.executor.memory	512m
yarn.scheduler.minimum-allocation-mb	384	spark.driver.memory	512m
yarn.scheduler.maximum-allocation-mb	1424		

- 執行時間：約 70 分鐘。
- Executor 記憶體只有 512 MB。
- YARN 單一 Container 最大只能申請 1,424 MB。
- Spark 可使用的記憶體太少，容易增加資料交換、垃圾回收與磁碟讀寫時間。

```
real    70m8.282s
user    0m49.700s
sys     0m4.144s
```


首次優化：調升配置與執行成果

參數調整細節

- 將 YARN 單一 Container 從 1,424 MB 提高到 6,144 MB。
- 將 Executor 記憶體從 512 MB 提高到 2,560 MB
- 加入 Executor 數量、CPU Core 與 Memory Overhead 設定。

配合 YARN 提升，同步調整 Spark 參數：

yarn-site.xml 參數	初始設定值	修改後設定值
yarn.nodemanager.resource.memory-mb	12288 (12G)	12288(12G)
yarn.nodemanager.resource.cpu-vcores	8	8
yarn.scheduler.minimum-allocation-mb	384	384
yarn.scheduler.maximum-allocation-mb	1424	→ 6144 MB

首次優化：調升配置與執行成果

參數調整細節

- 將 YARN 單一 Container 從 1,424 MB 提高到 6,144 MB。
- 將 Executor 記憶體從 512 MB 提高到 2,560 MB
- 加入 Executor 數量、CPU Core 與 Memory Overhead 設定。

配合 YARN 提升，同步調整 Spark 參數：

spark-default.conf 參數	初始設定值	修改後設定值
spark.yarn.am.memory	1024m	1024m
spark.yarn.am.memoryOverhead	x	512m
spark.executor.memory	512m	→ 2560M
spark.executor.memoryOverhead	x	1024m
spark.driver.memory	512m	→ 1024m
spark.driver.memoryOverhead	x	512m
spark.executor.instances	x	→ 3
spark.executor.cores	x	→ 2

首次優化：調升配置與執行成果

參數調整細節

- 將 YARN 單一 Container 從 1,424 MB 提高到 6,144 MB。
- 將 Executor 記憶體從 512 MB 提高到 2,560 MB
- 加入 Executor 數量、CPU Core 與 Memory Overhead 設定。

51%

運算時間大幅縮短

(70 分鐘 → 34 分鐘)

```
real    34m11.362s
user    0m50.486s
sys     0m3.934s
```

配合 YARN 提升，同步調整 Spark 參數：

spark-default.conf 參數	初始設定值	修改後設定值
spark.yarn.am.memory	1024m	1024m
spark.yarn.am.memoryOverhead	x	512m
spark.executor.memory	512m	→ 2560M
spark.executor.memoryOverhead	x	1024m
spark.driver.memory	512m	→ 1024m
spark.driver.memoryOverhead	x	512m
spark.executor.instances	x	→ 3
spark.executor.cores	x	→ 2

執行 histogram.py 面臨的挑戰

追求更高算力，卻遭遇中斷

為了應付更複雜的聚合運算，在執行第二階段 ETL 時，我們再度提升資源天花板。

- 調高 Executor Memory 4096 MB。
- 調高 Executor.instances 7 個 (每顆 2 核心)。

但卻引發了新問題：

結果：

程式執行超過兩小時仍未完成。

spark-default.conf 參數

spark.yarn.am.memory	1024M
spark.yarn.am.memoryOverhead	512m
spark.executor.memory	4096M
spark.executor.memoryOverhead	1024m
spark.driver.memory	1024m
spark.driver.memoryOverhead	512m
spark.executor.instances	7
spark.executor.cores	3

任務失敗的根本原因



被忽略的隱藏成本

每個 Executor 實際向系統索求的資源不是 4096 MB，而是包含 Overhead 在內的 **5120 MB**。



YARN 的資源分配機制

YARN 必須在「單一 NodeManager」上找到足夠的連續資源。它無法將叢集中多台主機剩下的「零碎記憶體」拼湊給同一個 Executor 使用。



單點負載嚴重超載

要求同時擠入 7 個耗費高記憶體的 Executor，總量已遠遠超出單一實體節點可穩定負荷的實體記憶體上限。

解決方式

spark-default.conf 參數	原始設定值	修改後設定值
spark.yarn.am.memory	1024m	1024m
spark.yarn.am.memoryOverhead	512m	512m
spark.executor.memory	4096m	4096m
spark.executor.memoryOverhead	1024m	1024m
spark.driver.memory	1024m	1024m
spark.driver.memoryOverhead	512m	512m
spark.executor.instances	7	6
spark.executor.cores	3	2

運算時間大幅縮短

2 個多小時未完成



2 分 52 秒

測試機 1 後續 ETL 展示

確立穩定的資源配置後，後續的資料處理流程皆能順暢發揮效能：

 Summary.py 運算階段

1 分 16 秒

高強度的資料聚合與摘要生成，在資源充足的情況下快速完成，毫無延遲。

 GeoJSON.py 轉換階段

41 秒

繁重的空間資料格式轉換，不到一分鐘即可產出可靠的分析結果。

實測機 2 實測結果

維持 YARN 相同的記憶體策略，但叢集設定從 8 核心倍增至 16 核心。我們將 Executor Cores 提高到 5：

```
66)      at org.apache.spark.sql.Dataset$.anonfun$ofRows$2(Dataset.scala:97)
        at org.apache.spark.sql.Session.withActive(Session.scala:827)
bigred@dtadm:~$ time spark-submit ~/elt/create_reliability_geojson.py
Traceback (most recent call last):
  File "/home/bigred/elt/create_reliability_geojson.py", line 26, in <module>
    summary = spark.table(SUMMARY_TABLE)
  File "/opt/zfs/sys/spark-3.4.4-bin-hadoop3/python/lib/pyspark.zip/pyspark/sql/session.py",
line 1476, in table
  File "/opt/zfs/sys/spark-3.4.4-bin-hadoop3/python/lib/py4j-0.10.9.7-src.zip/py4j/java_gateway.py",
line 1322, in __call__
  File "/opt/zfs/sys/spark-3.4.4-bin-hadoop3/python/lib/pyspark.zip/pyspark/errors/exceptions/captured.py",
line 175, in deco
pyspark.errors.exceptions.captured.AnalysisException: [REQUIRES_SINGLE_PART_NAMESPACE] spark_
catalog requires a single-part namespace, but got `hadoop_prod`.`tdcs`.

real    0m29.855s
user    0m14.321s
sys     0m1.542s

real    0m29.825s
user    0m14.094s
sys     0m1.591s
```


CPU 翻倍帶來的效能躍升對比



結論

為什麼 clean.py 效能差異有限？

- Spark 工作不一定全部都是 CPU 密集型。
- clean.py 包含大量檔案讀取、Shuffle、資料寫入與 Iceberg Commit。
- 這些工作可能受到 HDFS I/O、網路、磁碟速度、小檔案數量與資料分區影響。
- 因此 CPU 從 8 核心增加到 16 核心，不代表所有工作都會直接快兩倍。
- Histogram、Summary 等計算型工作，比較容易從更多 CPU Core 與平行 Task 中獲益。

⚠ 不是所有提升都來自 CPU

⚠ 優化是要讓 CPU、記憶體、Executor 數量、Shuffle 和檔案結構彼此配合。

Travel Time Calculation

旅行時間計算

旅行時間計算

輸入條件: 台北、新竹、星期一、14:00



解析門架序列

載入台北到新竹之間的所有門架



找出歷史日期

分別計算每個星期一的總旅行時間



動態加總整趟旅程的行程時間

從14:00的歷史資料開始查詢，加總時間達到五分鐘後，下一個路段查詢14:05的資料



回傳中位數總旅行時間

算出每個星期一的總旅行時間後，排序並回傳中位數、各路段行程秒數、門架經緯度

單機運算結果

單機運算 + 讀取CSV

- 運算時間: 467秒
- 讀取CSV檔所需時間過長

```
===== 最終結果 =====  
預估總旅行時間 (中位數):49.0 分鐘  
=====
```

程式執行完畢，總耗時：467.35 秒

```
{  
  "01F0248S_01F0264S": {  
    "seconds": 68,  
    "start_lon": 121.516603,  
    "start_lat": 25.077631,  
    "end_lon": 121.501386,  
    "end_lat": 25.077464  
  },  
  "01F0264S_01F0293S": {  
    "seconds": 115,  
    "start_lon": 121.501386,  
    "start_lat": 25.077464,  
    "end_lon": 121.472961,  
    "end_lat": 25.074797  
  },  
}
```

Spark 分散式運算

單機運算
+
CSV 檔



Spark 分散式運算
+
Parquet 分割儲存

問題 ①

第一次嘗試，執行時間為264秒

原因

共有20個路段、254個星期一需要查詢，每次查詢都需要進行join，引發大量shuffle

解決方案

在executor端做初步的資料篩選，一次性將資料collect回driver，建立字典提高後續的查詢速度

Spark 分散式運算

單機運算
+
CSV 檔



Spark 分散式運算
+
Parquet 分割儲存

問題 ②

driver端OOM

原因

將車種和出發時間的篩選留到collect之後，導致大量資料塞滿driver的記憶體

解決方案

將過濾條件設定在executor端，只回傳篩選後的結果給driver，並調高driver的記憶體配置

Spark效能測試

各參數組合結果比較

Executor	Core	Memory (G)	Result (seconds)
9	2	3	80
6	2	1	70
6	3	4	65
2	6	5	60
2	8	5	47



> 執行時間縮短 10 倍！

Visualization

視覺化

視覺化_使用工具



HTML
建立網頁結構



JavaScript
處理前端互動



Flask
串接前後端



CSS
網頁樣式設計



Bootstrap
提供 UI 元件與
版面工具



Leaflet
地圖視覺化

視覺化_網頁呈現

旅行時間查詢：
提供指定時間
的旅行時間查詢

穩定度分析：
提供歷史資料分析

 ReliRoute

旅行時間可靠度分析平台

 穩定度分析

 旅行時間查詢

 關於平台

© 2026 ReliRoute

TDCS Historical Traffic Data

讓歷史交通資料,幫助使用者判斷何時出發、哪裡容易塞

透過大數據分析,提供國道路段旅行時間與可靠度資訊,讓每一次出行都更安心、更有效率

 旅行時間查詢

 穩定度分析

平台特色



歷史資料分析

整合 TDCS 歷史交通資料,分析旅行時間與穩定度。



多維度分析

支援時段、日別、綜合分析,掌握不同情境下的路段表現。



視覺化呈現

透過互動式地圖與圖表,讓風險資訊一目了然。



可靠度評估

提供中位數、P95、緩衝時間與穩定度等級,輔助出行決策。

視覺化_穩定度分析

左側：
查詢條件

穩定度分析

依時段、日別或綜合條件,檢視國道路段旅行時間穩定度

穩定度分析

彙總層級

依日別+時段

時段

凌晨(00:00-05:59)

日別

平日

方向

南向

更新地圖



路段資訊

土城-樹林

03F0385S_03F0447S

中位數旅行時間 3 分 32 秒

P95 旅行時間 10 分 4 秒

緩衝時間 6 分 32 秒

穩定度等級 高風險

右側：
詳細資訊

中間：
地圖視覺化

視覺化_路段整理



視覺化_旅行時間計算

左側：
查詢條件

中間：
地圖視覺化

右側：
詳細資訊

旅行時間查詢

選擇起訖點與出發時間,查詢預估旅行時間

查詢條件

查詢模式

指定日期

起點

基隆

終點

高雄

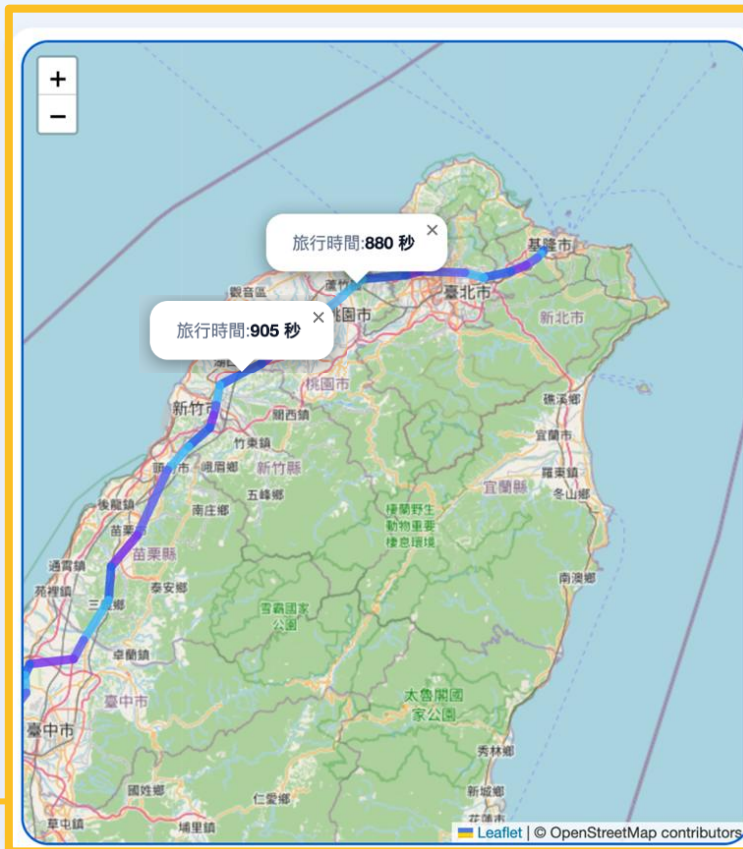
日期

2026/07/17

時間

下午05:08

開始查詢



總旅行時間

總旅行時間

479 分鐘

視覺化_SparkSession 建立方式優化



每次重新建立 Spark Session
平均 75.6 秒



Flask 啟動建立一次 SparkSession
API 重複使用 SparkSession

查詢等待時間：

75.6 秒 → 46.2 秒

縮短約
39%

常駐session

```
travel.js:208
▶ {start: 'Keelung', end: 'Kaohsiung', mode: 'weekday', time: '16:00', weekday:
'Monday'}
[*] 查詢耗時 49.53 秒 travel.js:214
travel.js:208
▶ {start: 'Keelung', end: 'Kaohsiung', mode: 'weekday', time: '23:30', weekday:
'Monday'}
[*] 查詢耗時 44.45 秒 travel.js:214
travel.js:208
▶ {start: 'Keelung', end: 'Kaohsiung', mode: 'weekday', time: '16:30', weekday:
'Monday'}
[*] 查詢耗時 44.83 秒 travel.js:214
```

平均 46.2 秒

非常駐session

```
travel.js:208
▶ {start: 'Keelung', end: 'Kaohsiung', mode: 'weekday', time: '16:00', weekday:
'Monday'}
[*] 查詢耗時 76.26 秒 travel.js:214
travel.js:208
▶ {start: 'Keelung', end: 'Kaohsiung', mode: 'weekday', time: '23:30', weekday:
'Monday'}
[*] 查詢耗時 80.88 秒 travel.js:214
travel.js:208
▶ {start: 'Keelung', end: 'Kaohsiung', mode: 'weekday', time: '16:30', weekday:
'Monday'}
[*] 查詢耗時 69.77 秒 travel.js:214
```

平均 75.6 秒

未來發展方向

1 | 即時資料更新與動態監測

串接交通即時資料來源，每 5 分鐘擷取最新車流與旅行時間資料，持續更新歷史資料庫，自動化計算P50、P95 與 Buffer Index，建立可即時反映路況變化的旅行時間可靠度監測平台。

2 | 多因素整合與短期預測

整合天氣、交通事故、道路施工、節假日及大型活動等外部因素，建立旅行時間預測模型，預估未來行車時間。

由「歷史資料查詢」進一步發展為「即時監測與預測」，協助交通管理單位提前掌握高風險路段，並提供用路人、導航平台與物流業者更即時且準確的行程規劃依據。

Thanks

謝謝聆聽